

COMPILADOR ACCESIBLE

Documentacion Tecnica

Angel Quispe Suarez

Universidad Nacional de San Agustin de Arequipa
Escuela Profesional de Ciencias de la Computacion
Curso: Compiladores — 2026

1. Índice

1.	Introduccion	3
2.	Arquitectura del Sistema	4
3.	El Lenguaje Accesible (.acc)	5
4.	Gramatica Formal (EBNF)	7
5.	Analizador Lexico (Fase 1)	9
6.	Analizador Sintactico (Fase 2)	12
7.	Arbol de Sintaxis Abstracta (AST)	14
8.	Deteccion y Reporte de Errores	16
9.	Uso del Compilador	17
10.	Ejemplos Completos	18
11.	Conclusiones	20

2. Introduccion

La ensenanza de la programacion enfrenta barreras significativas que van mas alla del dominio conceptual de los algoritmos. Los lenguajes de programacion tradicionales como C, Java o Python emplean una sintaxis cargada de simbolos cr pticos: llaves { } para delimitar bloques, barras verticales | | para la disyuncion logica, puntos y comas ; para terminar instrucciones, y operadores como && o != cuyo significado no resulta intuitivo para hablantes de espanol.

Esta carga simbolica representa una barrera especialmente severa para estudiantes con discapacidades sensoriales. Para un alumno con baja vision o ceguera que utiliza un lector de pantalla, distinguir entre { y } , (y) , o entre = y == es una tarea cognitivamente agotadora que distrae de la comprension del problema real que se intenta resolver. Para un alumno sordo cuyo primer idioma es la Lengua de Senas, la gramatica del espanol escrito ya supone un reto; anadir encima una segunda "lengua" de simbolos matematicos eleva el umbral de entrada de manera desproporcionada.

2.1. Magnitud del Problema

Estudios recientes sobre aprendizaje de programacion arrojan cifras preocupantes:

- Entre el **50 y 73 por ciento** del codigo fuente producido por estudiantes en sus primeros semestres *no compila* debido a errores de sintaxis.
- Solo el **2 por ciento** de los profesionales tecnologicos reporta algun tipo de discapacidad, frente al **19 por ciento** de la poblacion general, lo que evidencia una exclusion sistematica desde la etapa educativa.
- Los mensajes de error de compiladores clasicos suelen ser tecnicos y en ingles, lo que agrava la situacion para estudiantes hispanohablantes con discapacidad.

2.2. La Solucion Propuesta

El **Compilador Accesible** es un proyecto academico desarrollado en el marco del curso de Compiladores de la Universidad Nacional de San Agustin. Su objetivo central es construir un compilador completo para un lenguaje de programacion basado en espanol natural, eliminando los simbolos cr pticos y reemplazandolos por palabras clave familiares.

El lenguaje resultante, de extension **.acc** (por *accesible*), permite escribir programas como:

```
definir area como base por altura
si area es mayor que 100 entonces
    mostrar "Area grande"
sino
    mostrar "Area normal"
fin si
```

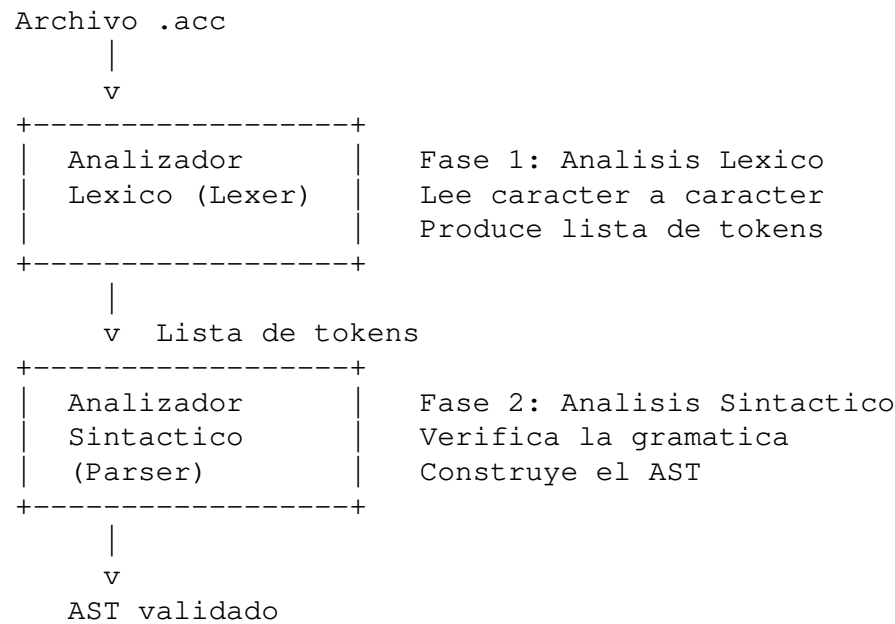
Este codigo es legible por cualquier hispanohablante sin conocimientos previos de programacion formal. La propuesta no es un reemplazante de los lenguajes profesionales, sino una *puerta de entrada* que reduce la carga cognitiva en la fase inicial del aprendizaje.

3. Arquitectura del Sistema

El compilador implementa un **pipeline de dos fases** que transforma un archivo de texto fuente **.acc** en un Arbol de Sintaxis Abstracta (AST) completamente validado. La arquitectura sigue el diseno clasico de un compilador de vanguardia frontal (*front-end*) desacoplado de cualquier generacion de codigo, lo que permite extenderlo hacia fases de analisis semantico y generacion en futuras iteraciones.

3.1. Pipeline de Procesamiento

El flujo de datos sigue la secuencia:



3.2. Estructura de Archivos

El proyecto sigue una organizacion modular clara:

Archivo	Responsabilidad
src/token.hpp, token.cpp	Definicion de tipos de token y estructura Token
src/lexer.hpp, lexer.cpp	Analizador lexico: texto -> lista de tokens
src/ast.hpp, ast.cpp	Nodos del Arbol de Sintaxis Abstracta
src/parser.hpp, parser.cpp	Analizador sintactico: tokens -> AST
src/main.cpp	Punto de entrada, manejo de argumentos CLI
ejemplos/	Programas de ejemplo .acc (basico, completo, etc.)
Makefile	Reglas de compilacion con g++

3.3. Tecnologia Utilizada

El nucleo del compilador esta implementado en **C++ estandar (C++17)** sin dependencias externas, lo que facilita la compilacion en cualquier plataforma con g++ o clang++ disponibles. Se hace uso extensivo de la biblioteca estandar: `std::string` , `std::vector` , `std::unordered_map` y herencia polimÃ³rfica para los nodos del AST.

4. El Lenguaje Accesible (.acc)

4.1. Principios de Diseño

El lenguaje **.acc** fue diseñado respetando los siguientes principios:

1. **Palabras clave en español natural:** Toda la sintaxis emplea vocabulario castellano cotidiano. No existe ningún símbolo de puntuación obligatorio.
2. **Insensible a mayúsculas:** El lexer normaliza todo a minúsculas antes de clasificar, por lo que `Definir`, `DEFINIR` y `definir` son equivalentes.
3. **Estructura por sangría semántica y palabras de cierre:** Los bloques se delimitan con palabras de cierre explícitas (`fin si`, `fin repetir`, `fin mientras`) en lugar de llaves, lo que es más amigable con lectores de pantalla.
4. **Un concepto, una línea:** Cada instrucción ocupa una línea, simplificando el análisis y los mensajes de error.
5. **Operadores aritméticos como palabras:** `mas`, `menos`, `por`, `entre` reemplazan a `+`, `-`, `*`, `/` respectivamente.

4.2. Construcciones del Lenguaje

A continuación se presentan todas las construcciones soportadas con ejemplos:

Declaración de Variables

```
definir nombre como 5
definir mensaje como "Hola mundo"
definir activo como verdadero
```

La palabra clave `definir` introduce una nueva variable; `como` asigna el valor de la expresión a la derecha. El tipo es inferido del literal.

Condición (si / sino / fin si)

```
si edad es mayor o igual que 18 entonces
    mostrar "Mayor de edad"
sino
    mostrar "Menor de edad"
fin si
```

La rama `sino` es opcional. El bloque de instrucciones termina con `fin si`.

Bucle Repetir (repetir N veces)

```
repetir 5 veces
    mostrar "iteracion"
fin repetir
```

Repite el bloque el número de veces especificado. La expresión de conteo puede ser cualquier expresión aritmética.

Bucle Mientras (mientras / fin mientras)

```
definir i como 0
mientras i es menor que 10
    definir i como i mas 1
fin mientras
```

Ejecuta el bloque mientras la condicion sea verdadera.

Salida de Datos (mostrar)

```
mostrar "El resultado es: "
mostrar area
mostrar base por altura
```

Comentarios

```
# Esto es un comentario, se ignora por el compilador
definir x como 10 # comentario al final de una linea
```

4.3. Operadores

Operadores de Comparacion:

Operador en .acc	Equivalente matematico
es igual a	
es mayor que	>
es menor que	<
es mayor o igual que	>=
es menor o igual que	<=

Operadores Aritmeticos:

Operador en .acc	Equivalente
mas	+
menos	-
por	*
entre	/

Operadores Logicos:

Operador en .acc	Significado
y	Conjuncion logica (AND)
o	Disyuncion logica (OR)
no	Negacion logica (NOT)

5. Gramatica Formal (EBNF)

La gramatica completa del lenguaje **.acc** se define a continuacion en notacion EBNF. Los símbolos entre comillas dobles son terminales (tokens). El asterisco * indica cero o mas repeticiones; el signo de interrogacion ? indica elemento opcional.

5.1. Reglas de Produccion

```

programa          -> instruccion*

instruccion        -> declaracion
                   |  condicional
                   |  bucle_repetir
                   |  bucle_mientras
                   |  salida

declaracion        -> "definir" IDENTIFICADOR "como" expresion NEWLINE

condicional        -> "si" condicion "entonces" NEWLINE
                   |  instruccion*
                   |  ("sino" NEWLINE
                   |  instruccion*)?
                   |  "fin" "si"

bucle_repetir      -> "repetir" expresion "veces" NEWLINE
                   |  instruccion*
                   |  "fin" "repetir"

bucle_mientras     -> "mientras" condicion NEWLINE
                   |  instruccion*
                   |  "fin" "mientras"

salida             -> "mostrar" expresion NEWLINE

expresion          -> termino (("mas" | "menos") termino)*

termino            -> factor (("por" | "entre") factor)*

factor             -> NUMERO
                   |  TEXTO
                   |  IDENTIFICADOR
                   |  "(" expresion ")"

condicion          -> ["no"] condicion_base
                   |  ("y" | "o") condicion_base)*

condicion_base     -> expresion comparador expresion

comparador         -> "es" "igual" "a"
                   |  "es" "mayor" "que"
                   |  "es" "menor" "que"
                   |  "es" "mayor" "o" "igual" "que"
                   |  "es" "menor" "o" "igual" "que"

```


5.2. Terminales

Los tokens terminales se definen con las siguientes expresiones regulares:

Terminal	Patron
NUMERO	<code>[0-9]+ ("." [0-9]+)?</code>
TEXTO	<code>["] (caracter \n \t)* ["]</code>
IDENTIFICADOR	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>
VERDADERO	<code>"verdadero"</code>
FALSO	<code>"falso"</code>
NUEVA_LINEA	<code>\n (salto de linea)</code>
COMENTARIO	<code># .* \n (descartado)</code>

Todos los identificadores y palabras clave son tratados con *case-insensitive matching* : el lexer convierte cada token a minúsculas antes de clasificarlo, de modo que `MOSTRAR` y `Mostrar` son equivalentes a `mostrar`

5.3. Ambigüedad y Precedencia

La gramática resuelve la ambigüedad de operadores aritméticos mediante la jerarquía clásica de la teoría de compiladores:

- **Menor precedencia:** `mas` y `menos` (aditivos) se manejan en la regla `expresion`
- **Mayor precedencia:** `por` y `entre` (multiplicativos) se manejan en la regla `termino` , que es llamada por `expresion`
- **Maximo nivel:** Los átomos (números, textos, identificadores y expresiones entre parentesis) se manejan en `factor`

Esta estratificación garantiza que `2 mas 3 por 4` se interprete como `2 + (3 * 4) = 14` y no como `(2 + 3) * 4 = 20`

6. Analizador Lexico (Fase 1)

6.1. Funcion y Responsabilidades

El analizador lexico, implementado en `src/lexer.hpp` y `src/lexer.cpp`, es el primer componente del pipeline de compilacion. Su tarea es leer el archivo fuente **caracter** por **caracter** y agrupar secuencias de caracteres en unidades significativas denominadas **tokens**

Concretamente, el lexer se encarga de:

- Eliminar espacios en blanco y tabulaciones irrelevantes.
- Reconocer y descartar comentarios (lineas que comienzan con `#` o comentarios al final de linea).
- Identificar palabras clave del lenguaje.
- Reconocer identificadores de variables.
- Reconocer literales numericos (enteros y decimales).
- Reconocer literales de texto entre comillas dobles, con soporte para secuencias de escape (`\n`, `\t`).
- Emitir tokens de nueva linea (`NUEVA_LINEA`) que el parser necesita para delimitar instrucciones.
- Registrar y reportar errores lexicos con numero de linea y columna exactos.

6.2. Estructura del Token

Cada token producido por el lexer es una instancia de la estructura `Token` definida en `src/token.hpp`:

```
struct Token {
    TipoToken tipo;    // Categoria del token (enum)
    string      valor;  // Lexema exacto del codigo fuente
    int         linea;  // Linea donde aparece (base 1)
    int         columna; // Columna donde comienza (base 1)
};
```

6.3. Categorias de Tokens

El enum `TipoToken` clasifica todos los tokens posibles:

Categoría	Tokens
Palabras clave	DEFINIR, COMO, SI, ENTONCES, SINO, FIN
Palabras clave (cont.)	REPETIR, VECES, MIENTRAS, MOSTRAR
Comparacion	ES, MAYOR, MENOR, IGUAL, QUE, A, O
Aritmetica	MAS, MENOS, POR, ENTRE
Logicos	Y, O_OP, NO
Literales	NUMERO, TEXTO, VERDADERO, FALSO
Identificadores	IDENTIFICADOR
Puntuacion	PAREN_IZQ, PAREN_DER
Control	NUEVA_LINEA, FIN_ARCHIVO
Errores	ERROR

6.4. La Clase Lexer

La clase principal `Lexer` expone la interfaz:

```
class Lexer {
public:
    Lexer(const string& codigo_fuente);
    vector<Token> tokenizar();    // Metodo principal

private:
    string fuente;
    int pos, linea, columna;
    unordered_map<string, TipoToken> PALABRAS_CLAVE;

    char actual();
    char siguiente();
    void avanzar();
    Token leerNumero();
    Token leerTexto();
    Token leerPalabraOIdentificador();
    void saltarComentario();
};
```

El metodo central es `tokenizar()`, que itera sobre el texto fuente en un bucle principal y, dependiendo del caracter actual, delega a uno de los metodos especializados de lectura.

6.5. Algoritmo de Tokenizacion

El algoritmo principal sigue el patron de *automata de estados finito*:

1. Leer el caracter actual.
2. Si es un espacio o tabulacion: avanzar y continuar.
3. Si es un salto de linea (`\n`): emitir token `NUEVA_LINEA`, incrementar contador de linea.
4. Si es `#`: llamar a `saltarComentario()` y continuar.
5. Si es un digito: llamar a `leerNumero()`

6. Si es " : llamar a leerTexto()
7. Si es (o) : emitir token de parentesis correspondiente.
8. Si es letra o guion bajo: llamar a leerPalabraOIdentificador()
9. De lo contrario: emitir token de tipo ERROR con el caracter desconocido.

6.6. Reconocimiento de Palabras Clave

El metodo leerPalabraOIdentificador() lee todos los caracteres alfanumericos consecutivos, los convierte a minusculas y busca el resultado en el mapa hash PALABRAS_CLAVE. Si hay coincidencia, se emite el tipo de palabra clave correspondiente. En caso contrario, se emite un token IDENTIFICADOR. Esta estrategia centraliza el reconocimiento de palabras reservadas y facilita agregar nuevas palabras clave en un solo lugar del codigo.

6.7. Ejemplo de Tokenizacion

Dado el siguiente fragmento de codigo fuente:

```
definir contador como 1
```

El lexer produce la siguiente secuencia de tokens:

Tipo	Valor	Linea	Columna
DEFINIR	definir	1	1
IDENTIFICADOR	contador	1	9
COMO	como	1	18
NUMERO	1	1	23
NUEVA_LINEA	\n	1	24

6.8. Tratamiento de Literales Especiales

Numeros Decimales: El metodo leerNumero() detecta la presencia de un punto decimal y continua leyendo digitos tras el, produciendo un unico token NUMERO con valor "3.14"

Cadenas con Secuencias de Escape: leerTexto() procesa el contenido entre comillas y sustituye la secuencia \n por salto de linea real y \t por tabulacion. Si la cadena no se cierra antes del fin del archivo, el lexer emite un token ERROR con el mensaje *cadena de texto no cerrada*

7. Analizador Sintactico (Fase 2)

7.1. Funcion y Responsabilidades

El analizador sintactico (*parser*), implementado en `src/parser.hpp` y `src/parser.cpp` , recibe la lista de tokens producida por el lexer y verifica que su estructura se ajuste a la gramatica formal del lenguaje **.acc** Si la estructura es valida, construye un **Arbol de Sintaxis Abstracta (AST)** que representa fielmente la jerarquia logica del programa.

Las responsabilidades concretas del parser incluyen:

- Confirmar que cada instruccion comienza con una palabra clave valida.
- Verificar que los bloques condicionales y de bucle cuenten con sus palabras de cierre (`fin si` , `fin repetir` , `fin mientras`).
- Verificar la presencia de palabras clave intermedias obligatorias (`entonces` , `veces` , `como`).
- Construir nodos del AST para cada construccion reconocida.
- Reportar errores sintacticos con ubicacion exacta (linea y columna).

7.2. Estrategia de Analisis: Descenso Recursivo

El parser implementa la tecnica de **descenso recursivo predictivo** (*recursive descent parser*) : cada regla de la gramatica se implementa como un metodo C++ independiente. El parser mantiene un puntero al token actual y avanza mediante el metodo `consumir()` , que verifica que el token actual sea del tipo esperado y avanza al siguiente. Si el tipo no coincide, se lanza una excepcion con el mensaje de error apropiado.

7.3. Correspondencia Gramatica-Metodos

La siguiente tabla muestra la correspondencia entre reglas gramaticales y metodos del parser:

Metodo C++	Regla Gramatical
<code>parsear()</code>	programa
<code>parsearInstruccion()</code>	instruccion (despachador)
<code>parsearDeclaracion()</code>	declaracion
<code>parsearCondicional()</code>	condicional
<code>parsearBucleRepetir()</code>	bucle_repetir
<code>parsearBucleMientras()</code>	bucle_mientras
<code>parsearMostrar()</code>	salida
<code>parsearExpresion()</code>	expresion
<code>parsearTermino()</code>	termino
<code>parsearFactor()</code>	factor
<code>parsearCondicion()</code>	condicion
<code>parsearComparador()</code>	comparador

7.4. Flujo del Metodo Principal

El metodo `parsear()` implementa la regla `programa -> instruccion*` :

```
NodoPrograma* parsear() {
    auto programa = new NodoPrograma();
    saltarNuevasLineas();
    while (!esFin()) {
        programa->instrucciones.push_back(
            parsearInstruccion()
        );
        saltarNuevasLineas();
    }
    return programa;
}
```

El metodo `parsearInstruccion()` actua como un *despachador* que examina el tipo del token actual y llama al metodo especifico:

```
NodoBase* parsearInstruccion() {
    switch (tokenActual().tipo) {
        case DEFINIR: return parsearDeclaracion();
        case SI:      return parsearCondicional();
        case REPETIR: return parsearBucleRepetir();
        case MIENTRAS: return parsearBucleMientras();
        case MOSTRAR: return parsearMostrar();
        default:
            error("Instruccion desconocida");
    }
}
```

7.5. Manejo de Bloques Anidados

Los bloques pueden anidarse a cualquier profundidad. El parser maneja el anidamiento naturalmente gracias al descenso recursivo: cuando `parsearCondicional()` llama a `parsearInstruccion()` en su cuerpo, y esa instruccion resulta ser otro `si`, se realiza una nueva llamada recursiva a `parsearCondicional()`. El retorno de la recursion despila el contexto correctamente.

8. Arbol de Sintaxis Abstracta (AST)

8.1. Que es el AST

El **Arbol de Sintaxis Abstracta** es una representacion jerarquica en memoria de la estructura logica de un programa. A diferencia del arbol de parseo concreto, el AST omite los detalles de la gramatica que no tienen relevancia semantica (como los tokens de nueva linea o las palabras de cierre), conservando unicamente la informacion estructural necesaria para las fases posteriores del compilador.

8.2. Jerarquia de Nodos

Todos los nodos del AST heredan de la clase base abstracta `Nodo`, que declara el metodo virtual puro `imprimir()` y el destructor virtual:

```
class Nodo {
public:
    virtual ~Nodo() = default;
    virtual void imprimir(int nivel = 0) const = 0;
};
```

La jerarquia completa de nodos es:

```
Nodo (clase base abstracta)
|
+-- NodoNumero           (literal numerico)
+-- NodoTexto            (literal de cadena)
+-- NodoBooleano         (verdadero / falso)
+-- NodoIdentificador    (nombre de variable)
+-- NodoOpBinaria        (operacion aritmetica: izq, op, der)
+-- NodoComparacion      (comparacion: izq, comparador, der)
+-- NodoOpLogica         (logica binaria: izq, y/o, der)
+-- NodoNegar            (negacion logica: operando)
+-- NodoDeclaracion      (nombre de variable, expresion)
+-- NodoCondicional      (condicion, bloque_si, bloque_sino)
+-- NodoBucleRepetir     (expresion_veces, cuerpo)
+-- NodoBucleMientras    (condicion, cuerpo)
+-- NodoMostrar          (expresion)
+-- NodoPrograma         (vector de instrucciones)
```

8.3. Ejemplo de AST Generado

Para el siguiente codigo fuente:

```
definir area como base por altura
```

El parser construye el siguiente arbol:

```

NodoPrograma
|
+-- NodoDeclaracion [nombre="area"]
|
+-- NodoOpBinaria [operador="por"]
|
+-- izquierda: NodoIdentificador [nombre="base"]
|
+-- derecha:  NodoIdentificador [nombre="altura"]

```

Para un programa mas completo con condicional:

```

si area es mayor que 40 entonces
    mostrar "Area grande"
fin si

```

El AST correspondiente es:

```

NodoPrograma
|
+-- NodoCondicional
|
+-- condicion: NodoComparacion [op="es mayor que"]
|
|   +-- izq: NodoIdentificador [nombre="area"]
|   +-- der: NodoNumero [valor="40"]
|
+-- bloque_entonces:
|   +-- NodoMostrar
|       +-- NodoTexto [valor="Area grande"]
|
+-- bloque_sino: (vacio)

```

8.4. Visualizacion del AST

El metodo `imprimir()` de cada nodo produce una representacion textual con sangria proporcional al nivel de profundidad del nodo. Esto permite al usuario del compilador invocar y obtener una vista legible del arbol completo, util para depuracion y verificacion pedagogica.

9. Deteccion y Reporte de Errores

9.1. Errores Lexicos

El analizador lexico detecta los siguientes tipos de error:

Error	Causa	Ejemplo
Caracter desconocido	Simbolo no perteneciente al lenguaje	@ # \$ %
Cadena no cerrada	Falta comilla de cierre antes del EOF	"hola mundo

9.2. Errores Sintacticos

El analizador sintactico detecta errores de estructura:

Error	Descripcion
Falta 'entonces'	Condicion sin la palabra clave 'entonces'
Falta 'como'	Declaracion sin la palabra clave 'como'
Falta 'veces'	Bucle repetir sin la palabra clave 'veces'
Instruccion desconocida	Token inicial no es palabra clave valida
Falta 'fin si'	Bloque condicional sin cierre
Falta 'fin repetir'	Bucle repetir sin cierre
Falta 'fin mientras'	Bucle mientras sin cierre

9.3. Formato de los Mensajes de Error

Todos los errores siguen el formato unificado:

```
[Linea X, Col Y] <descripcion del error>
<linea del codigo fuente>
<^--- indicador de posicion>
```

Ejemplo de error lexico:

```
[Linea 3, Col 15] caracter desconocido: '@'
definir x como @valor
^
```

Ejemplo de error sintactico:

```
[Linea 7, Col 1] se esperaba 'entonces' despues de la condicion
si edad es mayor que 18
^
```

Este formato, inspirado en los mensajes de compiladores modernos como clang, proporciona al estudiante toda la informacion necesaria para localizar y corregir el error: numero de linea, numero de columna, descripcion en espanol y visualizacion del contexto.

9.4. Recuperacion de Errores

En la version actual del compilador, la deteccion de un error detiene el procesamiento y reporta el primer error encontrado. Esta estrategia (*fail-fast*) es apropiada para un entorno de aprendizaje donde se desea que el estudiante corrija un error a la vez, evitando la cascada de mensajes de error que puede resultar confusa.

10. Uso del Compilador

10.1. Compilacion del Proyecto

El proyecto incluye un `Makefile` que automatiza la compilacion con `g++` :

```
make
```

Esto compila todos los archivos fuente de `src/` y produce el ejecutable `compilador` en el directorio raiz del proyecto.

10.2. Modos de Ejecucion

El compilador soporta los siguientes modos de operacion:

Comando	Descripcion
---------	-------------

10.3. Ejemplo de Salida en Modo Tokens

Para :

```
=== TOKENS ===
[1:1 ] DEFINIR      "definir"
[1:9 ] IDENTIFICADOR "contador"
[1:18] COMO        "como"
[1:23] NUMERO       "1"
[1:24] NUEVA_LINEA  "0"
[EOF ] FIN_ARCHIVO  ""
```

10.4. Ejemplo de Salida en Modo AST

Para :

```
=== ARBOL DE SINTAXIS ABSTRACTA ===
Programa
  Declaracion [base]
    Numero: 5
  Declaracion [altura]
    Numero: 10
  Declaracion [area]
    OpBinaria [por]
      Identificador: base
      Identificador: altura
  Mostrar
    Texto: "Calculando area del rectangulo"
  Mostrar
    Identificador: area
  Condicional
    Comparacion [es mayor que]
      Identificador: area
      Numero: 40
    Bloque-entonces:
      Mostrar
        Texto: "Area grande"
    Bloque-sino:
      Mostrar
        Texto: "Area pequena"
  ...
```

11. Ejemplos Completos

11.1. Programa completo.acc

El archivo ejemplos/completo.acc es el programa de referencia que ejercita todas las construcciones del lenguaje. Se presenta a continuacion con anotaciones:

```
# Ejemplo completo: usa todas las estructuras del lenguaje

# 1. Declaraciones de variables y operacion aritmetica
definir base como 5
definir altura como 10
definir area como base por altura

# 2. Salida de texto y valores numericos
mostrar "Calculando area del rectangulo"
mostrar area

# 3. Condicional con comparacion numerica
si area es mayor que 40 entonces
    mostrar "Area grande"
sino
    mostrar "Area pequena"
fin si

# 4. Bucle repetir con acumulador
definir suma como 0
repetir 5 veces
    definir suma como suma mas base
fin repetir
mostrar suma

# 5. Bucle mientras con condicion de parada
definir i como 10
mientras i es mayor que 0
    definir i como i menos 3
fin mientras
mostrar i

# 6. Condicion con operadores logicos compuestos
definir x como 7
si x es mayor que 5 y x es menor que 10 entonces
    mostrar "x esta entre 5 y 10"
fin si
```

11.2. Explicacion del Programa

El programa ilustra las seis construcciones principales del lenguaje:

Parte 1.

Declaraciones. Se crean tres variables: base con valor 5, altura con valor 10 y area

calculada

.ft R

como el producto de las dos anteriores. Muestra que una expresion aritmetica puede aparecer directamente como valor de una declaracion.

Parte 2.

Salida mixta. Se usa `mostrar` dos veces: una con un literal de texto y otra con una variable.

Parte 3.

Condicional completo. Se evalua si `area` (que vale 50) es mayor que 40; como la condicion es verdadera, se muestra "Area grande". La rama `sino` se incluye para demostrar la sintaxis completa.

Parte 4.

Bucle repetir. Se repite 5 veces la instruccion que acumula `base` en `suma`. Al terminar, `suma` vale 25 (5×5).

Parte 5.

Bucle mientras. Se decrementa `i` de 10 en pasos de 3 hasta que deja de ser mayor que 0. El valor final de `i` sera -2 ($10 - 3 - 3 - 3 = 1$, luego $1 - 3 = -2$).

Parte 6.

Logica compuesta. Se verifica que `x` (igual a 7) cumple simultaneamente dos condiciones: ser mayor que 5 y menor que 10. El operador `y` combina ambas comparaciones.

12. Conclusiones

El **Compilador Accesible** demuestra que es técnicamente viable construir un front-end de compilador completo para un lenguaje basado en español natural, con las siguientes características verificadas:

1. **Tokenización correcta del español natural.** El analizador léxico procesa palabras clave de varias palabras (como "es mayor o igual que"), literales decimales, cadenas con secuencias de escape y comentarios de línea, produciendo una secuencia de tokens limpia y precisa para todas las construcciones definidas.
2. **Validación estructural completa.** El analizador sintáctico de descenso recursivo verifica que la estructura del programa se ajuste estrictamente a la gramática formal definida en EBNF, garantizando la correctitud de cada instrucción antes de proceder.
3. **Construcción de un AST coherente.** El parser produce un Árbol de Sintaxis Abstracta con la jerarquía correcta de nodos, preservando la información necesaria para que fases posteriores (análisis semántico, generación de código) puedan operar sobre ella.
4. **Mensajes de error contextuales y precisos.** Cada error léxico o sintáctico incluye número de línea, columna, descripción en español y visualización del contexto, permitiendo al estudiante localizar y corregir el problema de manera autónoma.
5. **Gramática extensible.** El diseño modular del lexer (tabla hash de palabras clave) y el patrón de descenso recursivo del parser hacen que agregar nuevas palabras clave o construcciones requiera cambios mínimos y localizados.
6. **Base para fases futuras.** La arquitectura actual establece una base sólida para implementar: un *analizador semántico* que verifique tipos y variables declaradas, una *tabla de símbolos* para gestionar alcances, y un *generador de código* que produzca código intermedio o nativo.

En términos de impacto educativo, el proyecto confirma la hipótesis inicial: un lenguaje de programación basado en palabras del español cotidiano reduce significativamente la carga cognitiva asociada a la sintaxis, permitiendo que los estudiantes—incluidos aquellos con discapacidades sensoriales—se concentren en el aprendizaje de los conceptos algorítmicos en lugar de batallar con símbolos crípticos.

El **Compilador Accesible** no pretende reemplazar a los lenguajes profesionales, sino abrir una puerta de entrada más inclusiva al mundo de la programación, alineada con los principios del *diseño universal para el aprendizaje (DUA)* y con el compromiso de la Universidad Nacional de San Agustín con la equidad educativa.

Universidad Nacional de San Agustín de Arequipa — Curso de Compiladores — 2026

